

National University of Computer and Emerging Sciences
CS3006 - Parallel and Distributed Computing
Assignment 1 Report
Afnan Asif | 23L-0709

1. Part 1: Parallel Fractal Rendering

1.1 Implementation

The code is in `fractalWorker()` in `fractalThread.cpp`. We draw a 900x601 fractal. Each pixel maps to a complex number c and runs $z \leftarrow z^3 + c$ up to 300 times. Hard pixels need more math. Each thread owns different rows, so no locks are needed.

Naive mapping: each thread gets one block of rows. We set $\text{startRow} = (\text{height} * \text{threadId}) / \text{numThreads}$ and $\text{endRow} = (\text{height} * (\text{threadId} + 1)) / \text{numThreads}$ so every row is covered once, even when 601 does not divide evenly. Each thread also stores its own work time in `threadTimes[threadId]`.

Better mapping: thread i takes every N th row ($i, i+N, i+2N, \dots$). That mixes hard and easy rows. `FRACTAL_INTERLEAVE` chooses the policy (0 = naive, 1 = interleaved). We checked with `./fractal --sweep --check` on both views.

1.2 Results and graph

Table 1. Fractal speedup from `./fractal --sweep --check`.

Threads	Naive View 1	Naive View 2	Interleaved View 1	Interleaved View 2
1	0.99	1.00	0.99	1.01
2	1.96	1.16	1.95	2.00
3	2.26	1.79	2.88	2.92
4	2.97	2.20	3.81	3.80
5	2.92	2.74	4.70	4.53
6	4.40	3.28	5.63	5.68
7	5.11	3.43	5.45	5.37
8	5.62	4.04	7.56	7.41

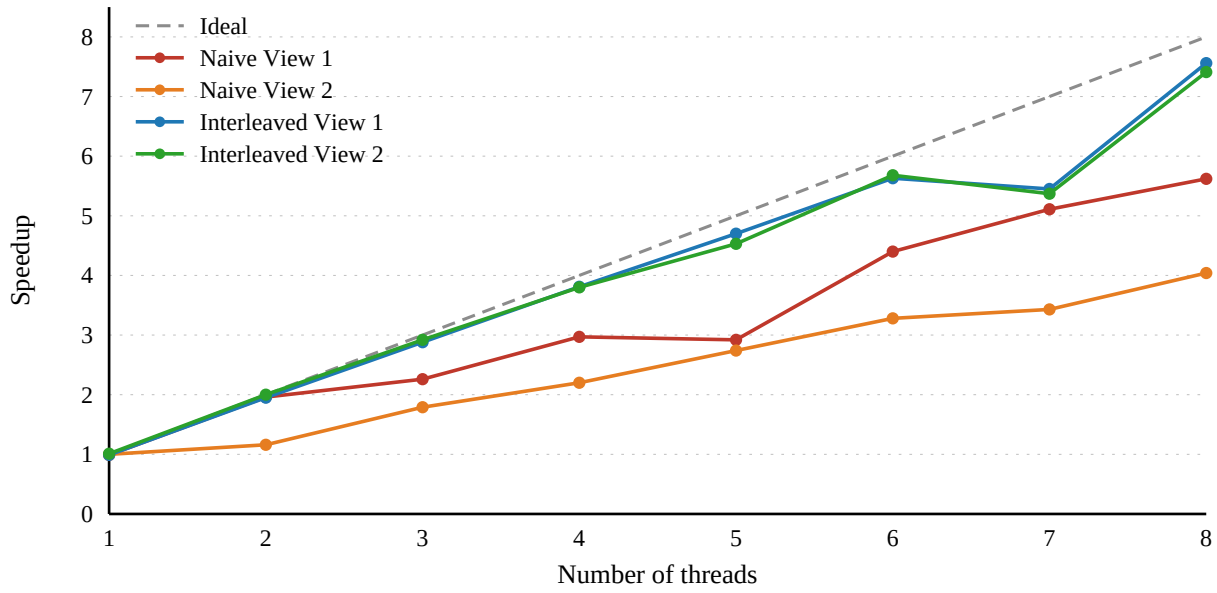


Figure 1. Fractal speedup vs thread count for both views.

Figure 1 and Table 1 show the same data. The grey dashed line is ideal linear speedup (2 threads $\rightarrow$ 2x, 8 threads $\rightarrow$ 8x). The red/orange curves are the naive split. They stay well below ideal: at 8 threads they only reach 5.62x (View 1) and 4.04x (View 2). The blue/green curves are interleaved rows. They stay much closer to ideal and reach 7.56x and 7.41x at 8 threads. Notice the dip at 3 threads on the naive curves: that is load imbalance, explained next.

1.3 Written analysis

Is speedup linear? No. With the naive split, speedup does not grow in a straight line. At 3 threads we got about 2.3x on View 1 and 1.8x on View 2. At 8 threads we got about 5.6x and 4.0x. Equal rows does not mean equal work, because some rows have many hard pixels and some have almost none.

What does the 3-thread point show? Load imbalance. Height is 601, so the blocks are about rows 0-199, 200-399, and 400-600. The whole run waits for the slowest thread.

Table 2. Per-thread work times, naive mapping, 3 threads.

Thread	Rows	View 1 time (s)	View 2 time (s)
0	0-199	0.1058	0.1428
1	200-399	0.1526	0.1100
2	400-600	0.1082	0.0064

Table 2 explains the 3-thread dip in Figure 1. On View 1, thread 1 owns the dense middle and takes 0.1526 s, while the others take about 0.106 s and 0.108 s, so speedup is only about 2.3x. On View 2, most hard pixels are upper, so thread 2 finishes in 0.0064 s and sits idle while thread 0 works 0.1428 s; speedup is only about 1.8x.

What did per-thread timings show? Finish time follows the slowest thread. After interleaved rows, the three View 1 times became almost equal: about 0.124 s, 0.124 s, and 0.121 s. So the earlier slowdown was imbalance, not a hard limit on parallelism.

What change for Part 4, and what speedup? We changed which rows each thread owns, not how many: interleaved rows i , $i+N$, $i+2N$, ... with no locks. Hard and easy rows get mixed. At 8 threads we got 7.56x on View 1 and 7.41x on View 2 (meets the 7-8x goal).

Did 16 threads help over 8? Only a little. Once work is balanced, extra threads add scheduling and cache overhead, so 16 is not much faster than a balanced 8.

2. Part 2: SIMD Clamped Alternating Series

2.1 Implementation

The code is in `clampedAltVector()` in `main.cpp`, using `PDCvector.h`. We process `VECTOR_WIDTH` elements at a time. A valid mask marks real lanes (needed for the last partial chunk). An alive mask tracks lanes that have not clamped yet. On each step, only alive lanes that still have work do a multiply (even step) or add (odd step). If a lane passes its cap, we set it to the cap and turn it off. Values under 0.0001 become exactly 0, same as serial. Output matched serial under `--check`.

2.2 Results

Table 3. Vector utilization from `./altseries -s 10000`.

VECTOR_WIDTH	Utilization
2	67.6%
4	62.9%
8	59.1%
16	56.2%

Table 3 is the key result for Part 2. Utilization falls as `VECTOR_WIDTH` grows: 67.6% at width 2, then 62.9%, 59.1%, and 56.2% at width 16. There is no speedup graph here because the assignment asks for utilization, not thread speedup.

2.3 Written analysis

Does utilization go up, down, or stay the same as width grows? It goes down.

Why? All lanes in one vector move together. Some finish early (small count or early clamp). Those lanes are clamped off and sit idle while other lanes in the same vector keep going. A wider vector packs more elements under one shared clock, so more lanes are likely already idle while a few slow lanes still work. The machine still pays for the full width, but fewer lanes do useful work each step. That is why utilization falls as width grows.

3. Part 3: AXPBY (Compute vs Memory Bound)

3.1 Implementation

The code is in `axpbyWorker()` in `axpbyThread.cpp`. It computes $\text{result}[i] = 2.5 * X[i] + (-1.5) * Y[i]$. Unlike Part 1, we must use contiguous blocks: thread `t` owns indices from $(N * \text{threadId}) / \text{numThreads}$ to $(N * (\text{threadId} + 1)) / \text{numThreads}$. Every index is covered once even when `N` does not divide evenly. Each thread writes a different part of result, so no locks are needed. Contiguous ranges also help memory streaming.

Correctness passed `--check` for `N = 1000003`, `8000001`, and `20000000`.

3.2 Results and graph

Table 4. AXPBY results from `./axpby -n 20000000 --sweep --check`.

Threads	Time (s)	GB/s	Speedup
1	0.0121	26.48	1.05
2	0.0092	34.68	1.37
3	0.0095	33.51	1.33
4	0.0097	32.98	1.30
5	0.0090	35.44	1.40
6	0.0094	34.16	1.35
7	0.0090	35.38	1.40
8	0.0090	35.65	1.41

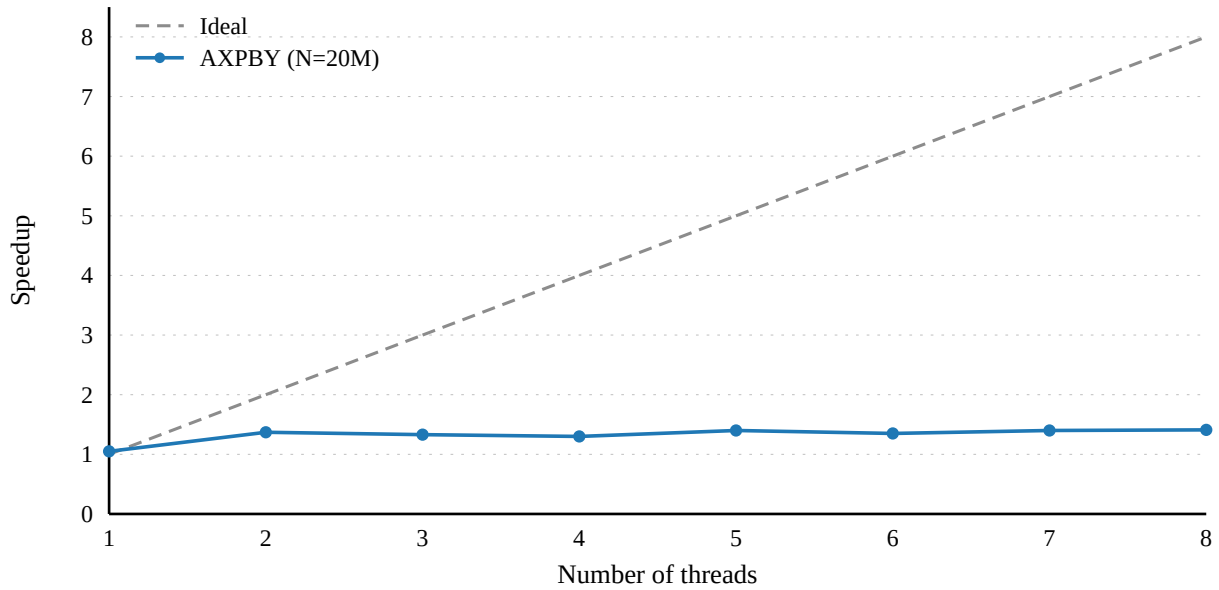


Figure 2. AXPBY speedup vs thread count.

Figure 2 and Table 4 show AXPBY scaling. The blue curve rises a little from 1 to 2 threads (about 1.05x to 1.37x), then stays flat near 1.3x to 1.4x through 8 threads. Bandwidth also plateaus around the mid-30s GB/s. Compared with Figure 1, where interleaved fractal reaches about 7.5x, this curve barely moves. That means extra threads are not the bottleneck anymore; memory is.

3.3 Written analysis

Which scales better, and why? The Part 1 fractal scales much better (about 7.5x at 8 threads) than AXPBY (about 1.3x to 1.4x). The reason is arithmetic intensity: how much math you do per byte moved. The fractal does up to 300 iterations per pixel, so it is compute-bound and extra cores stay busy. AXPBY does only a little math per element but must move a lot of data, so it is memory-bound. Once the memory bus is full, more threads help little.

Why count $4 \cdot N \cdot \text{sizeof}(\text{float})$ bytes when each element looks like 2 reads and 1 write? The reads are $X[i]$ and $Y[i]$; the write is $\text{result}[i]$ (three floats). The extra factor is write-allocate: before writing $\text{result}[i]$, the cache usually loads that line first, so result is counted as both a read and a write. That is two floats for X and Y plus two for result , or $4 \cdot N \cdot \text{sizeof}(\text{float})$ in total.